Tema 8

Optimización

Algoritmos genéticos

Índice

Esquema 2

Ideas clave 3

8.1 Introducción 3

8.2 Objetivos 4

8.3 Estructura de un algoritmo genético 4

8.4 Algoritmos genéticos sobre cadenas binarias 7

8.5 Operadores de Selección 7

8.6 Operadores de Cruce 10

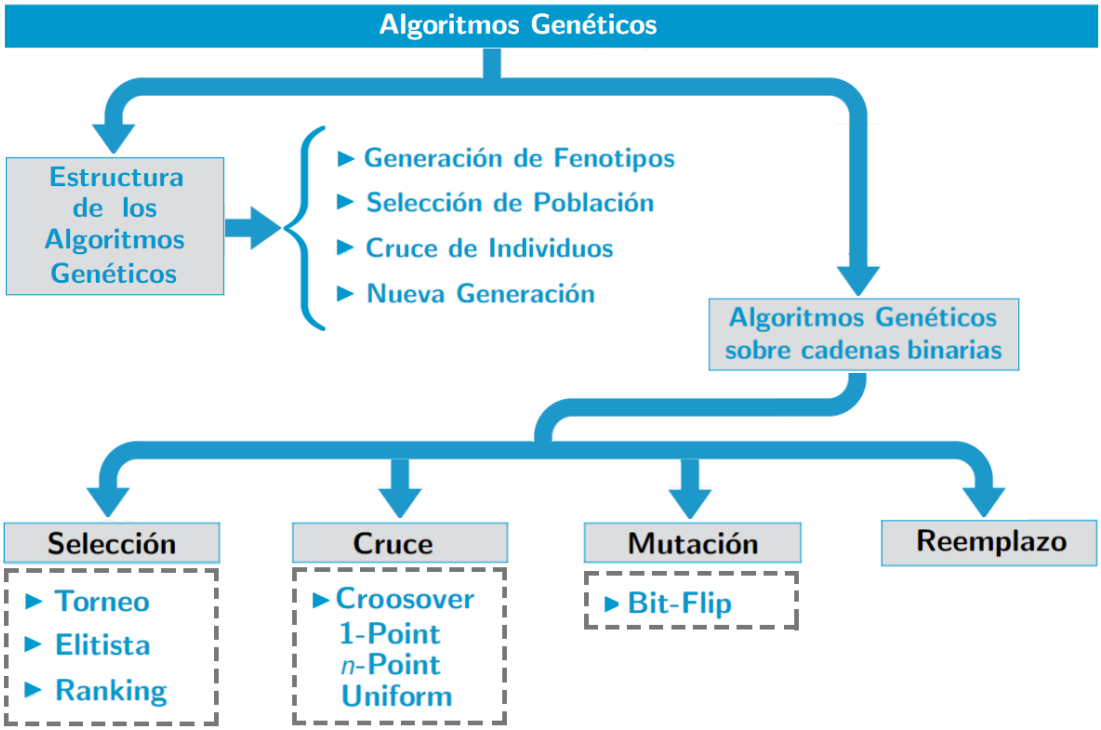
8.7 Mutación - Bit-Flip. 12

8.8 Selección de la nueva generación 12

8.9 El Problema de la Mochila 14

8.10 El problema del viajante 22

Esquema


Algoritmos Genéticos
Estructura de los Algoritmos Genéticos
Generación de Fenotipos
Selección de Población
Cruce de Individuos
Nueva Generación
Algoritmos Genéticos sobre cadenas binarias
Selección
Torneo
Elitista
Ranking
Cruce
Croosover
1-Point
n-Point
Uniform
Mutación
Bit-Flip
Reemplazo

Ideas clave

8.1 Introducción

Los algoritmos genéticos, fueron desarrollados en USA en 70's por: J. Holland, K. DeJong, D. Goldberg. Originalmente fueron aplicados a: Optimización discreta o combinatoria (binario).

Estos, son algoritmos de optimización que pueden ser pensados como métodos de búsqueda global. La idea del algoritmo es replicar el proceso de selección natural para encontrar las soluciones más aptas. Esto es, el algoritmo consiste en, dada una población de individuos, hacerla evolucionar mediante cambios aleatorios en sus genes y luego seleccionar a los mejores individuos de la población para que puedan perpetuarse.

En el marco de un problema de optimización, las posibles soluciones del problema conforman la población y la función de costo sirve para seleccionar a los individuos más aptos.

Hemos dicho que los cambios en los genes son aleatorios. Sin embargo, el grado de aleatoriedad está controlado (interviene en una pequeña parte del algoritmo). Así, los algoritmos genéticos son mucho más eficientes que las búsquedas generando soluciones completamente aleatorias y también que las búsquedas exhaustivas. Por otra parte, estos algoritmos no necesitan prácticamente ninguna hipótesis adicional del problema. Por ejemplo, no se ven afectados por la falta de derivabilidad (o continuidad) del problema.

8.2 Objetivos

Dentro de los objetivos principales, que podríamos destacar encontramos:

- Comprender la base de un algoritmo genético. Esto en términos de establecer la combinación que nos permita obtener la mejor elección de la población disponible y bajo los requerimientos dados.
- Determinar cada una de las etapas de un algoritmo genético y discriminar su uso de forma adecuada.
- Implementar los algoritmos genéticos en distintas situaciones, específicamente resolver el **Problema de la Mochila** y una aplicación al **Problema del Agente Viajero**.

8.3 Estructura de un algoritmo genético

Los algoritmos genéticos están inspirados en los procesos biológicos. Es por eso que se suele utilizar una terminología propia de la biología para designar los elementos que componen estos métodos. Podemos, pues, establecer una equivalencia entre la terminología que hemos venido utilizando hasta ahora y la nueva, la que hace referencia a los conceptos propios de la biología. Así, las funciones objetivo (o de costo) se llaman, en el contexto de los algoritmos genéticos, funciones de aptitud[1] (o simplemente aptitud), las soluciones candidatas para resolver el problema se llaman fenotipos y estas están codificadas a través de cromosomas.

Un conjunto de cromosomas determinan un **fenotipo**, de la misma manera que un conjunto de dígitos determinan un número. Evaluando los fenotipos en la función de aptitud, uno puede seleccionar qué candidatos son los que se adaptan mejor a su entorno. Por ejemplo, en un problema de minimización, aquellos fenotipos cuya imagen

[1] Comúnmente, la función de aptitud no coincide con la función objetivo. Es una modificación de la función objetivo adaptada para que verifique ciertas hipótesis.

sea menor por la función de aptitud. Cada iterado del algoritmo, produce una **nueva generación**. Esto es, los fenotipos tienen descendencia. En la nueva generación, se introducen cambios aleatorios en sus cromosomas, esto es, se producen **mutaciones**.

Siendo más específicos, cada fenotipo se codifica en un vector de parámetros numéricos (los cromosomas). La dimensión de este vector, supongamos que es N, es la que determina la dimensionalidad del problema. Esto es, un fenotipo g, vendrá codificado como

$$g = [p_1, p_2, \ldots, p_N].$$

Aquí, p_i es el valor numérico del cromosoma i. Es decisión del programador, elegir como se codifican las posibles soluciones de un problema en cromosomas. Por ejemplo, si uno quiere codificar un número real, es típico expresarlo mediante su base binaria. Así, cada solución se codifica como un vector de 1 y 0. Evidentemente, hay total libertad a la hora de elegir de qué manera se codifican los fenotipos. Dicho esto, una mala codificación, puede dar lugar a soluciones que no sean óptimas y un mal comportamiento del algoritmo.

Un algoritmo genético empieza seleccionando, de manera aleatoria, una población inicial de fenotipos. Entonces, se evalúa la aptitud de cada uno de los fenotipos. Consideremos algunos ejemplos simples y tratemos de evaluar cuando pueden funcionar como función de aptitud.

Ejemplo 1. Funciones de aptitud

1. Supongamos que tenemos el siguiente problema de optimización:

$$\text{maximizar } \; g(x) = -x^2 + 2x, \quad x \in [1, 2].$$

La función g es siempre positiva en el intervalo $[1, 2]$, así pues, podemos elegir la función de aptitud igual a la función objetivo: $f(x) = g(x)$.

2. Supongamos el siguiente problema de optimización:

$$\text{maximizar } \; g(x) = -x^2 + 4x, \quad x \in [1, 5].$$

La función g es negativa para $x > 4$. Su valor mínimo se alcanza en $x = 5$, donde toma el valor $g(5) = -5$. Sumando a g cualquier cantidad positiva mayor de 5 podemos asegurarnos una función de aptitud positiva. Por ejemplo, $f(x) = g(x) + 6$.

3. Supongamos, ahora, que queremos resolver el siguiente problema:

$$\text{minimizar } \; g(x) = x^2, \quad x \in [-10, 10].$$

Aquí el problema principal es que estamos tratando de minimizar o no de maximizar. La solución típica que se nos ocurriría, cambiar de signo g, resulta en una función no positiva. Esa función no positiva tiene alcanza su valor mínimo en $x = -10, 10$. En este caso, el valor mínimo es 100. Teniendo en cuenta esto, podemos elegir $f(x) = -g(x) + 101$.

Cuanto mayor sea la adaptación de un individuo al problema (mejor *fitness*: mayor en maximización, menor en minimización), mayor será la probabilidad de que el mismo sea seleccionado para reproducirse.

Esto seleccionará cuáles son los que mejor se adaptan para resolver el problema. El siguiente paso es crear un operador de selección que determine la probabilidad de que un subgrupo de fenotipos se reproduzcan en el siguiente iterado.

La población inicial se elige aleatoriamente, cuasialeatoriamente u obtener las soluciones iniciales a partir de otro heurístico o metaheurístico. La elección de población de manera aleatoria se puede realizar usando recursos en línea como: `https://www.augeweb.com/azar/`.

Resulta importante mantener la diversidad de la población y en este sentido, el tamãno N de la población repercute en el devenir de las nuevas generaciones, ya que,

poblaciones pequeñas corren el riesgo de no cubrir adecuadamente el espacio de búsqueda, mientras que poblaciones de gran tamaño pueden suponer excesivo coste computacional.

8.4 Algoritmos genéticos sobre cadenas binarias

En la descripción general del algoritmo genético, hemos dejado muchas cuestiones (que tienen que ver con operaciones genéticas) al aire, sin detallar, excusándonos en que dependían del problema. De hecho, parte de la utilidad de los algoritmos genéticos radica en la facilidad para adaptarse a distintas situaciones. En esta sección abordamos un caso concreto que es tremendamente importante: aquellos problemas en que hay que optimizar funciones de aptitud definidas en cadenas binarias. Esto es, los fenotipos de esta sección se codifican como un vector de unos y ceros:

$$g \in \{0, 1\}^N.$$

Como todo número real puede ser aproximado mediante una representación binaria finita, entonces los problemas de optimización de funciones de variables reales (con el número de variables que sea) se pueden reducir al tipo de problema que tratamos en esta sección. Vamos a ver primero, como implementar las operaciones genéticas con fenotipos codificados en cadenas binarias.

8.5 Operadores de Selección

Este operador también es elegido con libertad por el programador. Eso sí, es imprescindible que los fenotipos más aptos, tengan una mayor probabilidad de reproducirse, aunque la definición de más apto, pueda sufrir modificaciones. Veamos algunos operadores de selección.

Selección Proporcional al valor de la Función Objetivo - Elitista

Una manera de resolver esta cuestión es ponderar el resultado de la evaluación de aptitud. Por ejemplo, supongamos que queremos maximizar la función f y que ésta es no negativa. Podemos determinar la probabilidad de reproducción del fenotipo g_k con la siguiente fórmula:

$$p_k = P(g_k) = \frac{f(g_k)}{\sum_{k=1}^{N} f(g_k)}$$

p_k es la probabilidad de que el individuo k-ésimo sea seleccionado como padre y tomando N para denotar el número total de fenotipos. La hipótesis que hemos adoptado sobre la función de aptitud, que es no negativa, nos obligará a modificar la función objetivo, como se hizo en el **Ejemplo 1**. En este caso,

- Se muestrea tantas veces como sea necesario.
- Se puede reconsiderar el reemplazamiento.
- En general, esta selección converge a los superindividuos.

Notemos que, después de resolver el problema de optimización para la función de aptitud, tendremos que traducir la solución al problema de optimización original deshaciendo el cambio.

Selección por Muestreo Universal Estocástico -Baker 1987 - Ruleta

Se consideran sectores circulares proporcionales al valor de función objetivo de cada individuo (inverso si estamos minimizando). En este caso, se realiza un único giro de la ruleta.

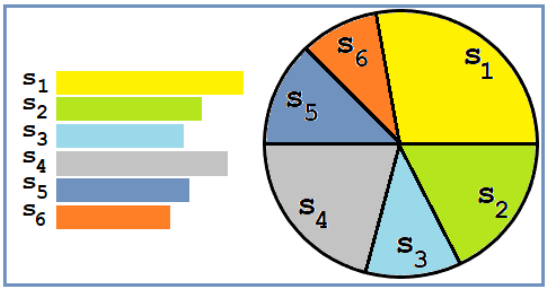


Figura 1: Selección por Ruleta

Los individuos son seleccionados a partir de marcadores igualmente espaciados y con comienzo aleatorio. Esto suele resultar en convergencias más lentas, pero es más sencilla de programar. El problema, la rápida convergencia a superindividuos.

Selección Proporcional al Ranking del individuo

La idea aquí, es justamente evitar problemas de convergencia a superindividuos basando las probabilidades de selección en fitness relativas, en lugar del absoluto. Por tanto, se asigna la probabilidad inversamente proporcional a su posición en el ranking, donde el ranking puede ser: lineal, exponencial, etc.

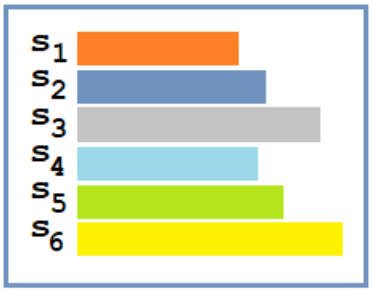


Figura 2: Selección por Ranking

Selección por Torneo

En este caso, se elege el tamaño k del torneo (habitualmente $k = 2$). Luego, se escoge al azar k individuos de la población con o sin reemplazamiento.

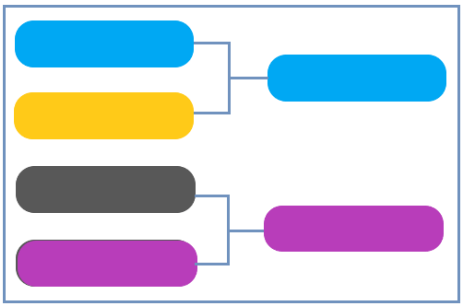


Figura 3: Selección por Torneo

Finalmente, la idea consiste en seleccionar el mejor individuo de este conjunto, y repetir el proceso hasta que tengamos el número de individuos deseado para ser progenitores.

Una se vez se ha calculado la aptitud de los distintos fenotipos de una población, hay que calcular los individuos de la siguiente población.

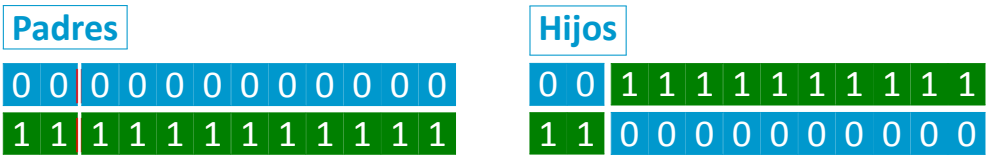
8.6 Operadores de Cruce

Asumiendo que se necesitan dos individuos para procrear y generar un entrecruzamiento cromosómico. Esto es, se produce, a partir de un par de fenotipos, otro que comparte muchos cromosomas de sus fenotipos paternos. La manera en que aparejar los distintos fenotipos, puede variar según el problema.

Podría considerarse como una condición **indispensable** para que el algoritmo funcione, es que aquellos fenotipos que tengan un mejor valor de la función de adaptación tengan más posibilidades de aparejarse. Una cuestión importante es que, para mantener el número de individuos constantes a lo largo de las sucesivas generaciones, se asume que cada pareja dará lugar a dos hijos. Una vez se ha generado la siguiente generación, el primer iterado se ha completado. Para ello, se pueden hacer uso de distintos operadores.

One-Point Crossover

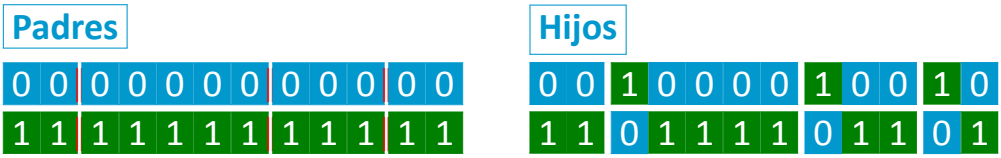
Este operador consiste en elegir una posición aleatoria común a ambos padres, para luego, dividir a los padres en este puntoy crear dos hijos copiando la cabecera de los cromosomas padres e intercambiando las colas.



Para este caso, se ha elegido la posición $n = 2$.

n-Point Crossover

Este operador consiste en elegir n posiciones aleatorias común a ambos padres. Luego, dividir a los padres en estos puntos y crear dos hijos copiando alternativamente las partes de los padres.



Para este caso, se han elegido las posiciones: $n = 2$, $n = 7$ y $n = 10$.

Uniform Crossover

Para el primer hijo, se cambia cada gen con probabilidad 0.5. El segundo hijo será el complementario del primero. Luego, se crean dos hijos copiando alternativamente las partes de los padres.

Padres

0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1

Hijos

0	1	0	0	1	0	0	0	1	0	1	0
1	0	1	1	0	1	1	1	0	1	0	1

8.7 Mutación - Bit-Flip

Es aquí donde se introduce la mutación aleatoria de algunos de los cromosomas. Se altera cada gen independientemente con probabilidad de mutación p_m. Habitualmente p_m se toma entre 1/tamaño de población y 1/longitud de cromosoma. Conviene que p_m sea baja, ya que si es demasiado alta, se pierde la información de los progenitores.

Hijo del Cruce

0	0	0	0	0	0	0	0	0	0	0	0

Hijo Mutado

0	0	1	0	0	0	0	0	1	0	0	0

En este caso, sólo dos genes mutaron al cumplir con la probabilidad p_m establecida. Una vez se ha generado la siguiente generación, el primer iterado se ha completado.

8.8 Selección de la nueva generación

Téngase en cuenta que teníamos un número de padres y hemos obtenido tantos descendientes. Debemos elegir N para pasar a la siguiente iteración.

Opciones:

- **Reducción simple:** quedarse con los descencientes y descartar los progenitores.
- **Reducción elitista:** quedarse con los mejores.

Se sigue evaluando la aptitud de la nueva generación y chequeando alguna de las nuevas soluciones es suficientemente buena. En caso contrario se vuelve a calcular una nueva generación de individuos. Es prudente, a efectos prácticos, que el criterio de parada no solamente se base en si alguna de las nuevas soluciones es suficientemente buena. En efecto, puede ser que los nuevos individuos no mejoren la aptitud de los progenitores o que, simplemente, la convergencia sea demasiado lenta. Para resolver esta cuestión se puede fijar un número máximo de iteraciones y terminar el programa en cuanto se alcanza ese número independientemente de la aptitud de la última generación.

Reemplazo

Después de cada iterado, podemos aplicar la estrategia del **reemplazo**. El último paso de cada generación consiste en reemplazar los elementos con peores aptitudes por nuevos individuos creados de forma aleatoria. Esto permite introducir sangre nueva en el sistema para ayudar a explorar el sistema y evitar que el sistema quede atascado en un óptimo local.

Así, la estructura general de un algoritmo genético es la siguiente:

1. Se produce una generación inicial de fenotipos.
2. Se calcula la aptitud de cada uno de los individuos.
3. Se aparejan los fenotipos según su aptitud.
4. Se calcula la nueva generación y se evalúa su aptitud.
5. Si no se satisface el criterio de parada, se vuelve al punto **3**.

8.9 El Problema de la Mochila

Se desea colocar objetos en una mochila, de forma tal que se maximice el valor de los objetos sin exceder su capacidad máxima.

Parámetros a definir

Se debe definir lo que es una **solución** y la función objetivo:

- **Solución** Una solución es un vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, donde $\mathbf{x} = \{0, 1\}^n$. En consecuencia,

$$x_i = \begin{cases} 1 & , \text{ objeto en la mochila} \\ 0 & , \text{ objeto } \textbf{no} \text{ está en la mochila} \end{cases}$$

- **Restricción --→ Conjunto de Factibilidad**

  ❶ Sea C_{max} la capacidad máxima de la mochila.

  ❷ Definimos el espacio de solución como:

$$\Omega = \left\{ [x_1, x_2, \ldots, x_n] : x_i \in \{0, 1\} \text{ y } \sum_{i=1}^{n} p_i x_i \leq C_{max} \right\}$$

- **Función Objetivo --→ Fitness**

  ❸ La **función objetivo** a maximizar, está definida por:

$$\boxed{f(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} x_i v_i}$$

Ejemplo 2. El problema de la mochila

Tenemos una mochila que tiene una capacidad de carga de 20 kg. Y un conjunto de paquetes que queremos llevar en ella. Cada paquete tiene un valor en euros y

un peso. determinado por la tabla siguiente:

Problema de la Mochila							
Paquete (i)	1	2	3	4	5	6	7
Peso (kg) (p_i)	3	4	4	6	3	6	5
Valor (€) (v_i)	2	5	3	1	3	5	7

Se pide determinar la combinación de paquetes de forma que quepan en la mochila y tengan el máximo valor posible.

Paso 1 **Fenotipos**

Se define un individuo como una posible combinación de objetos en la mochila. Para ello, usaremos un vector $\mathbf{b}$ de dimensión 7, cuyos valores pueden tomarse 1 o 0. Si $b_i = 1$ indicará que el objeto p_i se encuentra presente en la mochila, si $b_i = 0$ el objeto no se encontrará presente.

Paso 2 **Factibilidad**

Un individuo será factible siempre que:

$$P = \sum_{i=1}^{7} b_i p_i \leq 20$$

Iniciaremos el problema generando una población de n individuos factibles $\mathbf{b}_j$ de forma aleatoria.

Etapas del Algoritmo Genético

Iniciaremos el problema generando una población de n individuos factibles $\mathbf{b}_j$ de forma aleatoria. A continuación, seguiremos los pasos de los métodos genéticos:

1. **Selección de Padres :** Selección de elementos para cruzar a partir de las probabilidades γ_i. Para ello:

   a) Asociamos el valor del individuo j-ésimo como $V_j = \sum_{i=1}^{7} b_{j,i} v_i$.

   b) Tomamos el valor total de la población como $V_t = \sum_{j=1}^{n} V_j$.

   c) Definimos $\gamma_i = V_j / V_t$.

2. **Cruce de individuos:** Seleccionamos cuáles de los 7 objetos tomamos del primer individuo y cuáles del segundo de forma aleatoria. Si el nuevo individuo es no factible, generamos un nuevo individuo para añadir variabilidad a la población.

3. **Mutación:** Definimos la probabilidad p de mutación de un fenotipo ($b_{j,i}$). Por ejemplo 10^{-3}. Si un elemento es seleccionado, intercambiamos el valor de $b_{j,i}$.

4. **Remplazo:** Para finalizar la generación, ordenaremos los individuos de la generación anterior y la actual según su valor. Nos quedaremos con los n cuyo valor sea superior.

Ejemplo 3. El Problema de La Mochila - Completo

Tenemos una mochila que tiene una capacidad de carga de 7 kg. y un conjunto de paquetes que queremos llevar en ella. Cada paquete tiene un valor en euros y un peso. determinado por la tabla siguiente:

Problema de la Mochila					
Paquete (i)	1	2	3	4	5
Peso (kg) (p_i)	1	2	1.5	3	5
Valor (€) (v_i)	7	9	10	10	8

Se pide determinar la combinación de paquetes de forma que quepan en la mochila y tengan el máximo valor posible. Los parámetros para trabajar serán los siguientes:

- **Tamaño de la Población** $\boxed{N = 4}$

$$p_j = \{p_{j,1}, p_{j,2}, p_{j,3}, p_{j,4}, p_{j,5}\} \dashrightarrow \boxed{n = 5}$$

Generamos 4 vectores binarios de forma aleatoria.

- **Población Inicial** $\dashrightarrow$ **Aleatoria**
- **Función de selección de padres** $\dashrightarrow$ **Torneo** $\boxed{k = 2}$
- **Cruce** $\dashrightarrow$ **One Point Crossover**
- **Mutación** $\dashrightarrow$ **Bitsplitp** (Probabilidad de mutar $\frac{1}{2}$)
- **Selección de población para pasar a la siguiente generación.** $\dashrightarrow$ **Elitista** se eligen los 4 mejores.
- **Criterio de Parada** $\dashrightarrow$ **Dos generaciones** (dos iterados)

Población Inicial Buscamos 4 padres de partida de forma aleatoria. Por ejemplo, para seleccionar a los 4 padres, consideramos 4 muestras aleatorias de cinco números cada una en el intervalo $(0, 1)$, lo cual nos genera una probabilidad en cada artículo y la decisión de si consideramos el artículo o no, estará sujeta a si esa probabilidad iguala o supera el $40\,\%$.

Población Inicial $N = 4$					
p_1	**0.80**	0.13	**0.53**	**0.79**	0.35
p_2	0.01	0.11	0.29	0.31	0.23
p_3	0.09	**0.51**	0.17	**0.95**	0.33
p_4	**0.42**	0.15	**0.93**	0.11	0.03

Sabiendo que nuestra función de aptitud (**Fitness**) está dada por:

$$
\begin{aligned}
f(\mathbf{p}_j) &= \sum_{i=1}^{5} p_{j,i} v_i \\
&= p_{j,1} \cdot 7 + p_{j,2} \cdot 9 + p_{j,3} \cdot 10 + p_{j,4} \cdot 10 + p_{j,5} \cdot 8
\end{aligned}
$$

La población obtenida es justamente:

Población de Padres		Valoración de Padres	Posición
$\mathbf{p}_1$: $\boxed{1}\boxed{0}\boxed{1}\boxed{1}\boxed{0}$	--→	$f(\mathbf{p}_1) = 27$	$\boxed{1}$
$\mathbf{p}_2$: $\boxed{0}\boxed{0}\boxed{0}\boxed{0}\boxed{0}$	--→	$f(\mathbf{p}_2) = 0$	$\boxed{4}$
$\mathbf{p}_3$: $\boxed{0}\boxed{1}\boxed{0}\boxed{1}\boxed{0}$	--→	$f(\mathbf{p}_3) = 19$	$\boxed{2}$
$\mathbf{p}_4$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{0}$	--→	$f(\mathbf{p}_4) = 17$	$\boxed{3}$

Cruce De forma aleatoria, se eligen parejas dos a dos, para luego realizar el **One-Point Crossover** con $\boxed{n = 3}$. Toda esta información aleatoria, puede ser generada en: `augeweb.com/azar/`

Cruce Aleatorio		Mejor Padre		Hijos
$(2, 2)$	--→	$\mathbf{p}_2$: $\boxed{0}\boxed{0}\boxed{0}\boxed{0}\boxed{0}$	--→	$\mathbf{h}_1$: $\boxed{0}\boxed{0}\boxed{0}\boxed{1}\boxed{0}$
$(3, 3)$	--→	$\mathbf{p}_3$: $\boxed{0}\boxed{1}\boxed{0}\boxed{1}\boxed{0}$		$\mathbf{h}_2$: $\boxed{0}\boxed{1}\boxed{0}\boxed{0}\boxed{0}$
$(4, 4)$	--→	$\mathbf{p}_4$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{0}$	--→	$\mathbf{h}_3$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{0}$
$(2, 4)$	--→	$\mathbf{p}_4$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{0}$		$\mathbf{h}_4$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{0}$

Mutación Se generan en forma aleatoria números en $(0, 1)$ y podríamos establecer que si sale menor a 0.5 entonces no muta y si sale mayor, entonces cambia su valor.

Por tanto, por cada hijo $\mathbf{h}_j$, hay que generar cinco números aleatorios y ver si mutan entrada por entrada.

Mutación: ✔ ≥ 0.5 | ✖ ≤ 0.5					
$\mathbf{h}_1$	0.08	0.10	0.33	**0.89**	**0.53**
$\mathbf{h}_2$	0.01	0.11	**0.69**	0.31	0.23
$\mathbf{h}_3$	0.09	**0.51**	**0.77**	0.25	0.38
$\mathbf{h}_4$	0.42	**0.85**	**0.99**	0.11	**0.83**

En base a la tabla anterior, nos queda:

Hijos		Hijos Mutados
$\mathbf{h}_1$: 0 0 0 1 0	--→	$\mathbf{h}'_1$: 0 0 0 0 1
$\mathbf{h}_2$: 0 1 0 0 0	--→	$\mathbf{h}'_2$: 0 1 1 0 0
$\mathbf{h}_3$: 1 0 1 0 0	--→	$\mathbf{h}'_3$: 1 1 0 0 0
$\mathbf{h}_4$: 1 0 1 0 0	--→	$\mathbf{h}'_4$: 1 1 0 0 1

Ahora evaluamos a los hijos mutados en el Fitness:

Valoración de Hijos Mutados		Posición
$f(\mathbf{h}'_1) = \boxed{0} + \boxed{0} + \boxed{0} + \boxed{0} + \boxed{8} = 8$	--→	$\boxed{4}$
$f(\mathbf{h}'_2) = \boxed{0} + \boxed{9} + \boxed{10} + \boxed{0} + \boxed{0} = 19$	--→	$\boxed{2}$
$f(\mathbf{h}'_3) = \boxed{7} + \boxed{9} + \boxed{0} + \boxed{0} + \boxed{0} = 16$	--→	$\boxed{3}$
$f(\mathbf{h}'_4) = \boxed{7} + \boxed{9} + \boxed{0} + \boxed{0} + \boxed{8} = 24$	--→	$\boxed{1}$

Selección Ahora se seleccionan los cuatro mejores individuos de todos los individuos obtenidos y medidos:

Nueva Generación		Valoración		Posición
$\mathbf{p}_1$: 1 0 1 1 0	--→	$f(\mathbf{p}_1) = 27$	--→	$\boxed{1}$
$\mathbf{p}_3$: 0 1 0 1 0	--→	$f(\mathbf{p}_3) = 19$	--→	$\boxed{4}$
$\mathbf{h}'_2$: 0 1 1 0 0	--→	$f(\mathbf{h}'_2) = 19$	--→	$\boxed{3}$
$\mathbf{h}'_4$: 1 1 0 0 1	--→	$f(\mathbf{h}'_4) = 24$	--→	$\boxed{2}$

¡¡ Hemos obtenido la Primera Generación !!

Cruce Haremos **One-Point Crossover** con $\boxed{n = 1}$ para el primer cruce y $\boxed{n = 3}$ para el segundo cruce.

Cruce Aleatorio		Mejor Padre		Hijos
$(3, 2)$	--→	h_2': 0 1 1 0 0	$\xrightarrow{n=1}$	h_1: 0 1 0 0 1
$(3, 4)$	--→	h_4': 1 1 0 0 1		h_2: 1 1 1 0 0
$(4, 4)$	--→	h_4': 1 1 0 0 1	$\xrightarrow{n=3}$	h_3: 1 1 0 1 0
$(4, 1)$	--→	p_1: 1 0 1 1 0		h_4: 1 0 1 0 1

Mutación Para cada hijo h_j, se generan cinco números aleatorios y ver si mutan entrada por entrada.

Mutación: ✔ ≥ 0.5 | ✖ ≤ 0.5					
h_1	**0.68**	**0.70**	**0.93**	0.09	0.13
h_2	**0.91**	0.21	**0.79**	0.31	**0.53**
h_3	**0.99**	0.31	0.19	**0.75**	**0.88**
h_4	0.41	**0.55**	**0.59**	0.11	0.03

Luego,

Hijos		Hijos Mutados
h_1: 0 1 0 0 1	--→	h_1'': 1 0 1 0 1
h_2: 1 1 1 0 0	--→	h_2'': 0 1 0 0 1
h_3: 1 1 0 1 0	--→	h_3'': 0 1 0 0 1
h_4: 1 0 1 0 1	--→	h_4'': 1 1 0 0 1

Ahora evaluamos a los hijos mutados en el Fitness:

Valoración de Hijos Mutados		Posición
$f(h_1'') = \boxed{7} + \boxed{0} + \boxed{10} + \boxed{0} + \boxed{8} = 25$	--→	$\boxed{1}$
$f(h_2'') = \boxed{0} + \boxed{9} + \boxed{0} + \boxed{0} + \boxed{8} = 17$	--→	$\boxed{3}$
$f(h_3'') = \boxed{0} + \boxed{9} + \boxed{0} + \boxed{0} + \boxed{8} = 17$	--→	$\boxed{3}$
$f(h_4'') = \boxed{7} + \boxed{9} + \boxed{0} + \boxed{0} + \boxed{8} = 24$	--→	$\boxed{2}$

Selección Ahora se seleccionan los cuatro mejores individuos de ambas generaciones:

Nueva Generación		Valoración		Posición
$\mathbf{p}_1$: $\boxed{1}\boxed{0}\boxed{1}\boxed{1}\boxed{0}$	--→	$f(\mathbf{p}_1) = 27$	--→	$\boxed{1}$
$\mathbf{h}''_1$: $\boxed{1}\boxed{0}\boxed{1}\boxed{0}\boxed{1}$	--→	$f(\mathbf{h}''_1) = 25$	--→	$\boxed{2}$
$\mathbf{h}'_4$: $\boxed{1}\boxed{1}\boxed{0}\boxed{0}\boxed{1}$	--→	$f(\mathbf{h}'_4) = 24$	--→	$\boxed{3}$
$\mathbf{h}''_4$: $\boxed{1}\boxed{1}\boxed{0}\boxed{0}\boxed{1}$	--→	$f(\mathbf{h}''_4) = 24$	--→	$\boxed{3}$

¿Las soluciones obtenidas son factibles?

Veamos si satisfacen las restricciones de peso de la mochila. Recordemos nuestra función de peso:

$$\begin{aligned} P(\mathbf{p}_j) &= \sum_{i=1}^{5} p_{j,i} p_i \\ &= p_{j,1} \cdot 1 + p_{j,2} \cdot 2 + p_{j,3} \cdot 1.5 + p_{j,4} \cdot 3 + p_{j,5} \cdot 5 \end{aligned}$$

Entonces,

Valoración de Hijos Mutados		Solución
$P(\mathbf{p}_1) = \boxed{1} + \boxed{0} + \boxed{1.5} + \boxed{3} + \boxed{0} = 5.5$	$<$	7
$P(\mathbf{h}''_1) = \boxed{1} + \boxed{0} + \boxed{1.5} + \boxed{0} + \boxed{5} = 7.5$	$>$	7
$P(\mathbf{h}'_4) = \boxed{1} + \boxed{2} + \boxed{0} + \boxed{0} + \boxed{5} = 8$	$>$	7
$P(\mathbf{h}''_4) = \boxed{1} + \boxed{2} + \boxed{0} + \boxed{0} + \boxed{5} = 8$	$>$	7

Notas Importantes

1. Si hubiese dos o más soluciones menor que 7, se escoge la de menor peso.
2. Si todas fallan, no tiene sentido trabajar con un algoritmo donde las "soluciones" son **no** factibles y cada vez que esto ocurra se sale del algoritmo.
3. Se puede construir un algoritmo que en cada generación se evalúe el fitness y la restricción y por tanto, puede ir descartando y si no sirve, entonces se reemplaza. Esto puede resultar muy costoso, pero permite diseñar algoritmos para sólo factibles.

Problema con Penalizaciones

Sabemos que la función objetivo, está dada por:

$$\boxed{f(\mathbf{p}) = f(p_1, p_2, \ldots, p_n) = \sum_{i=1}^{n} p_i v_i}$$

la cual se puede penalizar de distintas maneras. Por ejemplo,

$$\boxed{f(\mathbf{p}) = f(p_1, p_2, \ldots, p_n) = \sum_{i=1}^{n} p_i v_i - L}$$

❶ En efecto, si se cumple que:

$$\sum_{i=1}^{n} p_i v_i < C_{\text{máx}} \quad \Rightarrow \quad L \equiv 0$$

❷ Si por el contrario:

$$\sum_{i=1}^{n} p_i v_i > C_{\text{máx}} \quad \Rightarrow \quad L \text{ es una constate muy grande}$$

esto para que el sistema la descarte. Entonces,

$$L = \begin{cases} 0 & , \quad \text{si } \displaystyle\sum_{i=1}^{n} p_i v_i < C_{\text{máx}} \\ 1000\left(\displaystyle\sum_{i=1}^{n} p_i v_i - C_{\text{máx}}\right) & , \quad \text{si } \displaystyle\sum_{i=1}^{n} p_i v_i > C_{\text{máx}} \end{cases}$$

8.10 El problema del viajante

En esta sección retomamos el problema del viajante, esto es, el problema central del tema 7. El enfoque de los algoritmos genéticos, nos permite resolver este problema

con un algoritmo distinto al que hemos empleado.

Recordamos que en el problema del viajante, tenemos que encontrar un camino Hamiltoniano óptimo en un grafo completo (en principio, dirigido o no). Para simplificar un poco las cosas, vamos a suponer que el grafo es no dirigido. Esto es, la distancia entre dos nodos del $d(v_i, v_j) = d(v_j, v_i)$ para cualquier par de vértices del grafo.

Una manera clara de codificar los fenotipos es mediante permutaciones ordenadas del conjunto de todos los vértices, esto es:

$$g = [\sigma(1), \ldots, \sigma(N)],$$

donde N es el número de vértices del grafo y $\sigma : \{1, \ldots, N\} \longrightarrow \{1, \ldots N\}$ es una aplicación biyectiva.

La función de costo dada por la distancia total recorrida a lo largo de un camino se puede aceptar como función de aptitud. Esto nos da una manera de evaluar la aptitud de cada uno de los fenotipos. Sin embargo, el cálculo de las nuevas generaciones y las modificaciones genéticas son más problemáticos en este ejemplo. En efecto, una solución solamente será válida si pasa exactamente una vez por cada uno de los vértices del grafo.

A continuación mostramos la Tabla 1, que ejemplifica, mediante el emparejamiento de dos caminos en un grafo de $N = 5$ vértices y como aplicar el tipo de cruce que hemos utilizado anteriormente resulta en un camino no Hamiltoniano.

Cruce En cada camino c_j, se realiza un cruce con el operador **One-Point Crossover** con $\boxed{n = 2}$.

	Progenitores		Descendientes
p_1:	3 5 | 1 2 4	--→	h_1: 3 5 5 3 2
p_2:	1 4 | 5 3 2	--→	h_2: 1 4 1 2 4

Tabla 1: Tabla de correspondiente a un emparejamiento prohibido en el problema. Los descendientes que tienen vértices repetidos representan un camino no Hamiltoniano.

La técnica que utilizaremos para solventar este problema se llama cruce cíclico. La idea es, partiendo de dos progenitores, elegimos una posición aleatoria de 1 a N. Tomemos por ejemplo: $\boxed{\mathrm{n} = 3}$, teniendo $\boxed{\mathrm{N} = 6}$

Progenitores

$\mathbf{p}_1$: 4 1 5 | 3 | 2 6

$\mathbf{p}_2$: 3 4 6 | 2 | 1 5

Intercambiamos los dos vértices correspondientes a esa posición en los vectores progenitores. Esto nos da dos descendientes que, a no ser que el vértice que hemos intercambiado sea el mismo, tendrán vértices repetidos.

En ese caso, intercambiamos el vértice duplicado del primer descendiente con el vértice que ocupe la misma posición en el segundo descendiente. Este proceso se repite hasta que los descendientes no tengan vértices repetidos. Ejemplificamos este proceso en la siguiente tabla.

Primer Descendiente

$\mathbf{h}_1$: 4 1 5 2 | 2 | 6

$\mathbf{h}_2$: 3 4 6 3 | 1 | 5

Observe que los tres primeros quedan intactos y al intercambiar el cuarto gen, también se intercambia el quinto gen, para así evitar que estos repitan.

Segundo Descendiente

$\mathbf{h}'_1$: 4 | 1 | 5 2 1 6

$\mathbf{h}'_2$: 3 | 4 | 6 3 2 5

Nota Nos hemos quedado con las colas que ya no repiten y nos ubicamos en los genes 1 y 2. Este proceso se repite hasta que los descendientes no tengan vértices repetidos.

Intercambiamos el segundo gen y obtenemos entonces un descendiente con una cola sin repetir vértices, por tanto falta intercambiar un sólo gen.

Tercer Descendiente

$\mathbf{h}_1''$: | 4 | 4 5 2 1 6

$\mathbf{h}_2''$: | 3 | 1 6 3 2 5

Finalmente intercambiamos el primer gen, para así completar el proceso.

Cuarto Descendiente

$\mathbf{h}_1'''$: 3 4 5 2 1 6

$\mathbf{h}_2'''$: 4 1 6 3 2 5

Mutación La mutación genética de los cromosomas es mucho más sencilla de llevar a cabo.

- Se escogen al azar dos cromosomas (vértices) de cada miembro de la nueva generación y se intercambian.
- La probabilidad de mutación, como en otros casos, tiene que ser baja.